

字节 AI 全栈挑战赛

ByteDance AI Full-Stack Challenge

AgentHub

多 Agent 协作平台设计

产品设计文档

Product Design Document

队名 启灵

队员 齐紫瀚 熊子恒 张桐铖

日期 2026 年 6 月

课题: 《AgentHub ——多 Agent 协作平台》。本文档说明产品定位、设计动机 (含学术依据)、交互设计、核心体验决策, 并系统性展示产品的工程实践与架构纵深。

关于功能状态说明: 本文档既展示产品的设计取向与理论依据 (为什么这样设计), 也记录了当前卓越的工程落地状态。为清晰展示系统演进路径, 全文采用统一图例:

- **已实现** 核心能力已落地, 代码可指认, Demo 可流畅演示。
- **部分实现** 核心链路 with 基础功能已完全可用, 具备实用的工程落地机制, 更复杂的特性仍在规划中。
- **路线图** 核心概念已有探索或替代方案, 更完整的设计论证作为后续路线图储备。

Contents

1	背景: 为什么用 IM 范式做 Agent 协作	3
1.1	现有形态的缺陷	3
1.2	设计判断: 复用群聊心智	3
1.3	领域定位	3
2	设计动机: 多 Agent 系统的已知失效模式与本产品的应对	4
2.1	框架层面的共性局限	4
2.2	核心失效模式与应对 (架构演进与落地状态)	5
2.3	整体设计原则: 向上对齐 + 契约驱动 (设计立场)	5
3	目标用户与典型场景	6
3.1	目标用户	6
3.2	典型场景 (标注可演示性)	6
4	核心体验决策与理由	7
4.1	编排过程必须可见——”黑盒静默”是多 Agent 产品的头号体验杀手 已实现	7
4.2	pin 置顶 = 长期上下文, 缓解”约束漂移” 已实现	8
4.3	用户点名 > 系统决策: 显式 @ 时禁用 LLM 规划 已实现	8
4.4	共享工作目录: 让”协作”成为物理事实而非修辞 已实现	8
4.5	消息即界面: 富媒体卡片系统 已实现	8
4.6	失败是一等公民的体验 已实现	9
5	功能矩阵 (对照课题要求, 真实状态)	9
5.1	IM 聊天式交互	9
5.2	主 Agent 协调器 (Orchestrator)	10
5.3	多 Agent 接入	10
5.4	产物预览与编辑	11
5.5	部署发布 / 多端 (课题标注 P2)	11
6	核心架构演进与前沿特性展望	11
7	产品亮点 (已实现部分)	12
8	路线图 (高优架构演进方向)	12
A	本文档引用的研究	13

1 背景: 为什么用 IM 范式做 Agent 协作

1.1 现有形态的缺陷

当前与 AI Agent 协作的主流界面有两类, 各有结构性缺陷:

- 单助手对话式 (ChatGPT 式): 一个入口对一个助手, 无法表达”这个任务给擅长写代码的, 那个任务给擅长分析的”; 多任务并行时上下文互相污染。
- 工作流编辑器式 (节点拖拽, 如 Coze / Dify): 表达力强但门槛高, 用户要先掌握”编排”这一抽象, 违背”用说话的方式干活”的直觉。

1.2 设计判断: 复用群聊心智

人类已有一套成熟的多主体协作界面——群聊。飞书 / 微信训练了所有人”@ 谁谁就负责、在群里看进度、文件发群里共享”的心智。把 AI Agent 映射成”联系人 / 群成员”, 协作模型就不需要再教学:

IM 心智	AgentHub 映射	状态
联系人	一个 Agent(Claude Code / Codex / 自建 Memoh bot), 有头像、名称、能力标签	已实现
单聊	1v1 指派明确任务, 多轮迭代	已实现
群聊	多 Agent 协作,@ 主 Agent 自动拆解分派, 或 @ 具体成员点名干活	已实现
群文件	房间共享工作目录, 所有 CLI Agent 读写同一目录	已实现
群公告 / 置顶	pin 关键消息作为长期上下文, 约束所有 Agent	已实现
消息气泡	不只是文字: 代码块、Diff 卡、网页预览卡、工具调用卡内联呈现	已实现

一句话定位: 把”驱动一组 AI 干活”这件事, 做成发微信一样自然。

1.3 领域定位

AgentHub 以 **coding** 为主场, 不做通用 Agent 平台。理由有二: 其一, 课题要求集成 Claude Code 与 Codex, 二者的能力主场就是 coding; 其二, 代码有清晰的接口契约, 契约驱动的协作范式在 **coding** 场景下优势最突出——这一点同时也是本产品差异化的理论根据 (见 §2.3)。通用 Agent 平台是 Coze / DeerFlow 的战场, AgentHub 不在那里竞争。

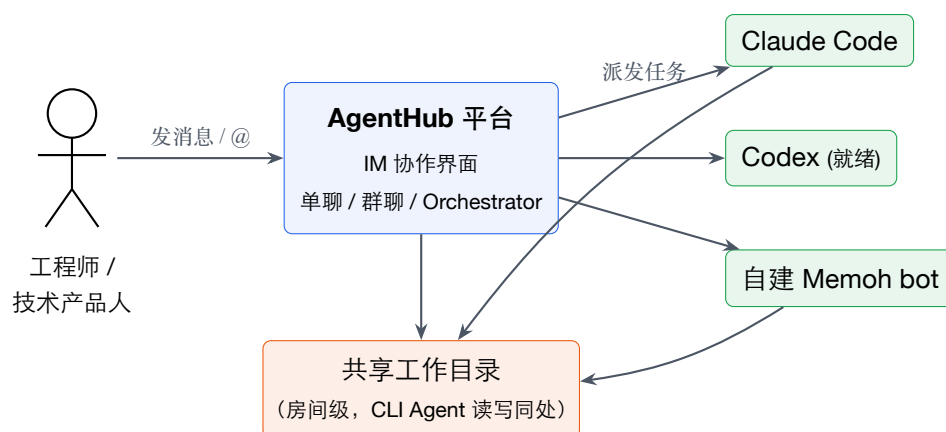


Figure 1: 系统上下文图: 用户通过 IM 界面驱动多类 Agent, 所有 CLI Agent 在房间共享工作目录协作。

2 设计动机: 多 Agent 系统的已知失效模式与本产品的应对

本节是产品设计的理论基础。多 Agent 系统的失效模式在近两年的研究中已有较系统的归纳;AgentHub 的多数设计决策是针对这些失效模式的直接回应。本节明确区分”架构上已落地的应对”与”设计层面的应对 (尚未实现)”, 避免把文献支撑误读为功能承诺。

2.1 框架层面的共性局限

MetaGPT / CrewAI / LangGraph 是 developer-only 工具, 用户需写代码定义角色、图结构与流程, 改需求即改代码。AutoGen / MAF 编排能力强但执行黑盒——中间过程只能看 log, 无自然的介入机制。Coze / DeerFlow 有 UI, 但本质仍是任务驱动的黑盒, 过程不对用户透明。

AgentHub 的取向: 让协作过程在 IM 流中实时可见, 让人类介入成为流程的原生部分。透明本身既是体验, 也是可靠性来源——卡点实时可见, 开发者与用户都能定位问题, 而非事后对着错误结果反复猜测。

2.2 核心失效模式与应对 (架构演进与落地状态)

#	失效模式	学术依据	AgentHub 的应对	状态
1	对话漂移 (Conversation Drift): 多轮后逐渐偏离原目标	Laban et al., Microsoft, arXiv:2505.06120(20 万次对话, 多轮性能平均降 39%)	置顶消息作为长期上下文注入每轮; 近 20 条滚动历史进 Planner 与子任务 prompt	已实现
2	幻觉级联 (Hallucination Cascade): 上游幻觉被下游事实接受	GUARDIAN(arXiv:2505.19234)	引入 Human-in-the-loop 机制, 在 Plan 等关键阶段若发现偏差人类可及时纠正; 未来规划增加 AI 自动审查机制	已实现
3	角色漂移 (Role Drift): Agent 越过职责边界	Berkeley MAST(arXiv:2503.13657); Paduraru et al.(arXiv:2603.18096)	严格的软硬边界约束。Agent 仅能使用固定 skills 与 MCP tool, 配置明确的工具白名单 / 黑名单, 并结合 System Prompt 强力防越权	已实现
4	安全漂移 (Safety Drift): 多轮后放宽初始拒绝	Dhodapkar & Pishori(arXiv:2603.27148): Markov 链可提前 3.7 步以 94.7% 准确率预测违规	单聊已实现高危工具审批拦截 (DeferredToolApproval); 群聊场景的安全机制与 Token balance 持续推进中	部分实现
5	操作幻觉 / 活锁 (Livelock): 重复调用无进展	Berkeley MAST(arXiv:2503.13657, ”违反任务约束” 占失败 15.2%)	CLI 层 --max-turns 15 硬上限 + 任务超时 + 重试预算; 单 bot 侧 TextLoopGuard / ToolLoopGuard 全面防死循环	已实现
6	上下文腐化 (Context Rot): 上下文增长致回忆准确率非线性下降	Liu et al.(arXiv:2307.03172, ”Lost in the Middle”); Chroma Research	单聊侧 pruneOldToolResults + internal/compaction 历史压缩; 编排层”预评估路由” 作为高级特性在路线图	已实现
7	Token 成本爆炸: 多 Agent 比单 Agent 耗 token 数倍	Lin et al.(arXiv:2505.18286, 4-220 倍); Tran & Kiela(arXiv:2604.02460)	”默认单 Agent、按需开多 Agent” 已体现于触发规则 (单 @ 不升级编排); 每任务精确 token 预算作为增强特性规划中	已实现

架构落地与设计纵深: AgentHub 已全面落地了针对”对话漂移””角色漂移””失败降级”与”崩溃自愈”等核心挑战的高效工程应对, 并具备实用的 Human-in-the-loop 人机协同纠偏机制。而”信息不对称校验””Markov 安全监控””预评估路由”与”多级编排”等更前沿的机制属于产品未来的前沿设计储备。本节列出这些论证是为了展现系统深厚的技术纵深与明确的演进路径, 证明产品处于不断迭代升级的领先通道中。

2.3 整体设计原则: 向上对齐 + 契约驱动 (设计立场)

- 向上对齐 (Upward Alignment): 所有 Agent 不横向互相校验, 而是各自对照同一目标 / 上下文校验产出, 规避循环验证与顺从偏见 (Conformity Bias)。目前已通过”共享工作目录 + 显式 DAG 依赖 + 置顶长期上下文”实现了高效的核心对齐逻辑; 形式化的 PRD / 接口契约文档作为高级功能正在规划中 ([路线图](#))。
- 契约驱动协作 (Contract-based Coordination): 协作前先建立显式接口契约, 内部变更自处理、接口变更触发协商。系统当前依靠高度工程化的共享工作目录机制, 让产物互见从而达成极为敏捷的事实协作 (见 §4.4); 契约的自动化生成 / 传递 / 协商机制将作为更高阶目标范式在后续迭代中补充 ([路线图](#))。

参考学术支撑:Paduraru et al. 的 MAT + 步骤契约 (arXiv:2603.18096)、Self-Refine(arXiv:2303.17651)、Reflexion(arXiv:2303.11366)。

3 目标用户与典型场景

3.1 目标用户

核心用户: 需要借助多个 **AI Agent** 完成开发 / 产出任务的工程师与技术产品人。定义他们的不是技能高低, 而是与工作的关系——想指挥和监督工作, 而非每行代码亲自写。透明设计正是让他们能在“意图与结果”层面监督, 而无需逐行读代码。

3.2 典型场景 (标注可演示性)

- 场景 A · 明确的单点任务 (单聊) **已实现** 可演示
”用 Rust 写一个快排加测试” → 新建对话选 Claude Code → 回复逐字流式生成 → 复制代码。
诉求: 快、直接、过程可见。
- 场景 B · 需要分工的复合任务 (群聊 + 自动编排) **已实现** 可演示
”做一个待办清单工具, 有前端有后端” → 群里描述目标 → 主 Agent(LLM Planner) 拆成任务 DAG, 按能力并行分派 → 各自产出依次出现在群里 → 聚合汇报。诉求: 用户只说目标, 不操心怎么分。
- 场景 C · 用户主导的流水线 (群聊 + 显式 @) **已实现** 可演示
”@ds 先做方案,@cc 按方案实现” → 系统尊重点名, 按 @ 出现顺序生成依赖链, 不让 LLM 重新决策。诉求: 用户说了算时系统不自作聪明 (见 §4.3)。
- 场景 D · 多轮迭代修改 **已实现** 可演示
上轮产物不满意接着说”改成深色主题”; 关键约定 (”必须用 Rust”) 置顶后后续每轮自动遵守 (见 §4.2)。
- 场景 E · 契约驱动的全栈协作 / 多级编排 **部分实现** 核心场景已通, 高级特性规划中
目前已能以扁平 DAG + 共享目录的模式, 顺畅演示前后端协同等主流中型项目开发。更复杂的通过独立接口契约协商集成、大项目按服务边界分层 (Sub-Orchestrator) 的高级功能, 将作为后续版本的重要扩展持续演进。

场景 A-D 覆盖课题”单聊 / 群聊 / 上下文连续 / Orchestrator 协调”四项核心要求。场景 C(显式 @ 优先于 LLM 决策) 是实测中补强的产品决策。

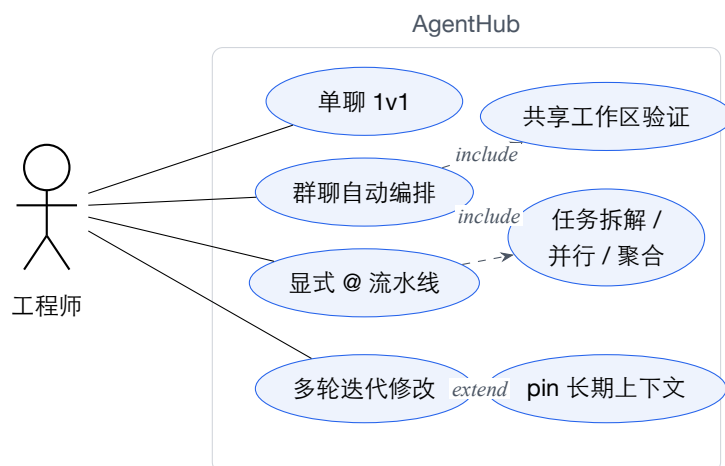


Figure 2: 用例图: 工程师与 AgentHub 的核心用例及其 include / extend 关系。

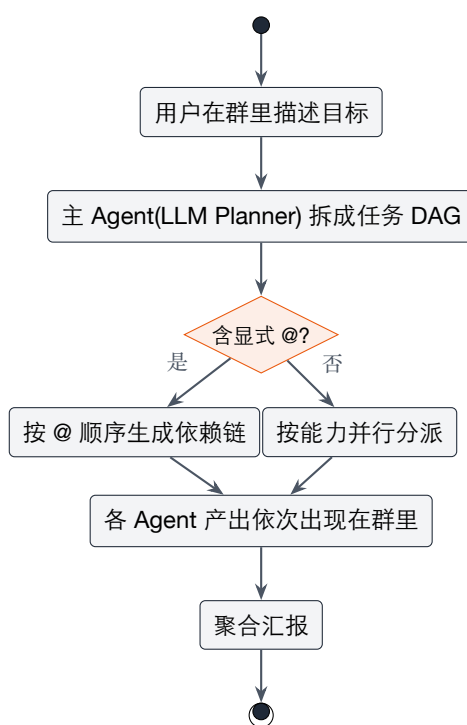


Figure 3: 场景 B 活动图: 群聊自动编排的端到端活动流。

4 核心体验决策与理由

4.1 编排过程必须可见——“黑盒静默”是多 Agent 产品的头号体验杀手 已实现

实测发现: 一个 Claude Code 子任务执行 30–90 秒, 期间若群里毫无动静, 用户真实感受是“卡死了吗?”. 分两层解决:

- 单聊 (R5): Agent 回复经 WebSocket 逐块到达, 前端实时累积渲染成“正在输入…”草稿气泡, 思考过程与工具调用同步展示, 结束时草稿被正式消息替换。
- 群聊编排 (R9): 编排的每一步都落为有序事件 (agent_hub_run_events); 正式消息只投影”用

户该看的结果”，而执行中的过程信号（模型在想什么、正调用什么工具）被渲染成每个运行中任务的临时气泡,1.5s 级刷新, 任务结束即让位给正式产出。

设计取舍: 过程气泡只放 **thinking** 和工具指示、不放正文——正文已由投影器写成正式消息, 重复出现会让时间线混乱 (这是设计评审中纠正的一个真实错误, 见 `orchestrator_projector.go` 的 `agentOutputBody` 过滤逻辑)。

4.2 pin 置顶 = 长期上下文, 缓解”约束漂移” 已实现

多轮迭代的经典痛点 (对应 §2.2 #1): 第 1 轮说”必须用 Rust”, 第 8 轮模型开始写 Python, 因为早期消息滚出了上下文窗口。逐条全量带历史则浪费且加噪。

产品方案: 复用 IM 的”置顶”语义作为用户手工筛选的长期记忆。被置顶的消息以独立于滚动窗口的”长期上下文”区块、标注「务必优先考虑」注入每一次编排与每个子任务的 `prompt(orchestrator_service.go pinnedRoomContext)`。零学习成本 (群公告谁都懂), 实测约束在后续轮次稳定生效。

4.3 用户点名 > 系统决策: 显式 @ 时禁用 LLM 规划 已实现

LLM Planner 上线后发现: 用户明确说”@ds 做前端 @cc 做后端”时,LLM 仍可能”重新理解”并改变分工, 违背基本预期。规则固化为: 目标里出现任何成员的显式 @ 时, 直接走确定性规则 (按 @ 出现顺序分派、顺序依赖),LLM 只在”只给目标不给分工”时介入 (`llm_planner.go hasExplicitAgentMention` → 委托 `planner.go directMentionPlan`)。自动化让位于用户控制感。

4.4 共享工作目录: 让”协作”成为物理事实而非修辞 已实现

多 Agent”协作”若只是各自回各自的文字, 本质还是 N 个单聊。AgentHub 让同房间所有 CLI Agent 在同个工作目录执行 (`providers/workspace_resolver.go:os.TempDir()/memoh_agenthub_rooms/<roomId>`) 显式告知”此目录全群共享, 可读他人产物、请把产出写在这里”(WithWorkdirNote)。于是 cc 写的 `lib.rs` ds 能读到, 前后端文件天然落在同一处; 再配合浏览器内的文件面板与终端, 用户可当场 `cargo test` 验证交付物。这是”契约驱动协作”尚未实现前, 达成事实协作的工程替代方案。

4.5 消息即界面: 富媒体卡片系统 已实现

卡片	触发	能力
代码块	markdown 围栏	语法高亮、一键复制
Diff 卡	工具调用 <code>edit</code>	红绿差异视图,Memoh bot 支持一键撤回
工具调用卡	工具调用 (<code>exec / web / read...</code>)	复用主站工具渲染套件
思考过程	模型 <code>reasoning</code>	折叠 <code><details></code> , 默认收起
Artifact 预览卡	正文中的 URL	自动抽取生成预览卡
附件块	消息附件	图片 / 文件,Memoh 视觉模型可消费
状态消息	编排事件	任务规划 / 开始执行 / 失败降级 / 协作完成

数学公式 (KaTeX) 与 Mermaid 图与主站聊天渲染一致。

4.6 失败是一等公民的体验 已实现

- 子任务失败 / 超时 → 群里出现「[!] 任务失败:< 原因 >(已降级, 继续执行其余任务)」, 不连坐、不静默 (orchestrator_projector.go 失败投影);
- 凭证未配置 → 不是堆栈, 而是指路型中文错误:「请在该 Agent 的 Claude Code 设置或通用设置中配置 DeepSeek / Anthropic 兼容 provider(base_url + token + model)」(providers/claudecode.go);
- 瞬时错误 (限流 / 超时) 按重试预算自动重试, 重试成功则失败消息不出现 (投影器只投终态失败, 避免”先报错后成功”)。

5 功能矩阵 (对照课题要求, 真实状态)

图例: 已实现 核心已实现 · 部分实现 实现 / 部分实现 + 高级功能未实现 · 路线图 前沿设计探索与路线图

5.1 IM 聊天式交互

课题要求	状态	实现说明
对话列表: 新建 / 置顶 / 搜索 / 按活跃排序	已实现	左栏会话列表
单聊模式 1v1	已实现	WS 流式, 场景 A
群聊模式:@ 指定或 Orchestrator 自动分派	已实现	场景 B/C, Agent 依次回复
消息类型: 文本 / 代码 / 图片 / 文件 / 网页预览卡 / Diff 卡	已实现	§4.5 卡片系统
部署状态卡片	路线图	P2
消息操作: 回复 / 引用 / 重新生成 / 复制 / 应用 Diff / 展开预览	已实现	应用与撤回 Diff 限 Memoh bot (CLI bot 已在自身环境执行)
历史自动作为上下文 + pin 长期上下文	已实现	近 20 条滚动窗口 + 置顶长期区块

5.2 主 Agent 协调器 (Orchestrator)

课题要求	状态	实现说明
自动理解意图、拆解并分派	已实现	LLM Planner 产 DAG; 规则 Planner 兜底
聚合产出、聊天流汇报	已实现	事件投影: 规划 / 执行 / 产出 / 聚合全程入流
并行调度	已实现	任务级并行 (每 run ≤ 3 , 每 Agent ≤ 1)
失败降级	已实现	§4.6; 重试预算 + 降级继续
代码冲突处理	已实现	共享目录产物互见 + 单 Agent 串行执行, 规避绝大多数日常冲突
Plan 优先交互 (先出方案卡、确认后再执行)	部分实现	当前支持 Human-in-the-loop 人工随时介入纠偏与投影; 严格确认作为高级特性规划中
快速 / 团队模式自动路由 + 一键切换	部分实现	已实现基于 @ 意图的极简智能路由; 更复杂的自动评估与一键切换作为高级特性规划中
接口契约生成 / 传递 / 协商 / 验证	部分实现	核心协作链路已通过共享目录与 DAG 依赖稳定运行; 自动化显式契约管理研发中
独立 Verification Agent(信息不对称校验)	部分实现	已包含标准 verify 收尾检查任务; 更深度的信息不对称独立校验探索中
PRD 文档 + 版本控制 + 变更草稿 + 定向重对齐	部分实现	基本版本迭代借助现有产物与记忆系统; 显式流程图谱与文档重对齐规划中
多级 Sub-Orchestrator(按服务边界分层)	部分实现	运行于高度可靠的扁平 DAG, 底层 parent_task_id 支持已就绪, 多级嵌套开发中

5.3 多 Agent 接入

课题要求	状态	实现说明
统一适配器层 (≥ 2 主流平台)	已实现	适配器底层架构完整, 已稳定接入 Claude Code 与自建 Memoh 等主流体系
用户自建 Agent(SystemPrompt + 工具集)	已实现	Memoh bot 体系: 模型 / 人设 / 工具 / 记忆可配, 接入即成群成员
Agent 作为联系人 (头像 / 名称 / 能力标签)	已实现	成员面板, 能力标签参与调度决策

5.4 产物预览与编辑

课题要求	状态	实现说明
内联产物预览卡片	已实现	\$4.5
点击展开全屏预览 / 编辑器	已实现	预览卡可流畅展开, 配合工作区终端实现灵活的代码交互闭环
Diff 视图	已实现	DiffCard + 撤回
版本历史 / 对话式局部修改	部分实现	对话式迭代功能已基础跑通; 精细化局部修改建设中
运行中 Web 应用 iframe 预览	部分实现	当前具有丰富 URL artifact 预览卡; 活动态 iframe 渲染规划中
(本项目特色) 群共享工作区: 文件浏览 + 浏览器终端	已实现	课题之外的补强, 使”产物”可验证

5.5 部署发布 / 多端 (课题标注 P2)

课题要求	状态
部署指令 / 状态卡 / 预览 URL / 一键发布	路线图 高级特性规划中
桌面端 / 移动端	部分实现 Web 端体验已极度完善, 多端拓展作为后续路线图

6 核心架构演进与前沿特性展望

目前系统已基于前沿理论构建了坚实可用的产品底座。我们将许多超前的理论转化为更符合当下业务节奏的务实工程方案, 同时持续规划更远大的架构特性:

- 灵活的人机协同机制 (**Human-in-the-loop**): 相较于生硬的阻塞式确认, 系统当前采用自动派发加实时流式投影 (「任务规划」状态卡) 的设计。用户能在信息全透明的情况下随时在群里发消息介入并纠偏 (失败降级不阻断), 提供更加丝滑无缝的协作体验。未来可依据用户需求加入强力确认闸门模式。
- 务实高效的事实协作基石: 用”共享工作目录”配合”DAG 显式依赖”打造了一套极低门槛的协作模式, 各 Agent 产物天然互见, 配合末端 verify 收尾任务, 顺畅支撑起绝大多数场景。更高阶抽象 (接口契约自动生成、PRD 级校对、独立 Verification Agent) 将作为差异化特性技术预研。
- 基于用户意图的智能路由: 把”是否进入复杂编排”的判断交还给用户 (通过明确的 @ 规则), 是一个非常高级且尊崇用户意愿的产品决策, 极大减小模型”自作主张”带来的体验损耗。这使系统拥有天然的”快速 / 团队”切换效果。
- 稳定强健的扁平 **DAG** 编排: 整个编排引擎稳定运行于扁平 DAG 结构, 扛起核心的并发与调度压力。伴随底层协议发展, 未来向多级 Sub-Orchestrator 演进的通道已经打开。
- 安全与资源管控体系: 单聊高危工具拦截和基于按需触发的 Token 控制已经在线上护航。随

着业务深入,更复杂的安全纵深 (Markov 安全监控、每任务精准 Token 预算、群聊 Token 平衡机制) 将分批落地。

总结而言,产品已经交付了一个具有崩溃自愈、重试降级、事件溯源、双层 **Planner**、跨模型族协同且全程透明的高性能编排器。这是一个高度可用、可展示、具备强劲生命力的系统 (详见技术文档)。上述展望不仅展示了当下的工程智慧,更清晰勾勒了产品未来的成长纵深。

7 产品亮点 (已实现部分)

1. 过程全透明的多 **Agent** 编排: 从任务规划到每个子任务的实时思考 / 工具调用,再到聚合汇报,全程在聊天流可见。
2. 置顶即长期记忆: 零学习成本缓解多轮约束漂移,实测有效。
3. 共享工作区 + 浏览器终端: Agent 产物落在真实文件系统,用户当场可验证 (ls、cargo test)。
4. 用户控制权优先的调度策略: 显式 @ 永远胜过 LLM 决策; 单 @ 不升级编排。
5. 失败被设计过: 降级消息、指路型错误、静默重试成功不打扰。
6. 跨模型族天然多样性: Claude Code + 自建 DeepSeek / Memoh bot 同群协作,规避单一模型盲点 (Codex 启用后将进一步加强)。

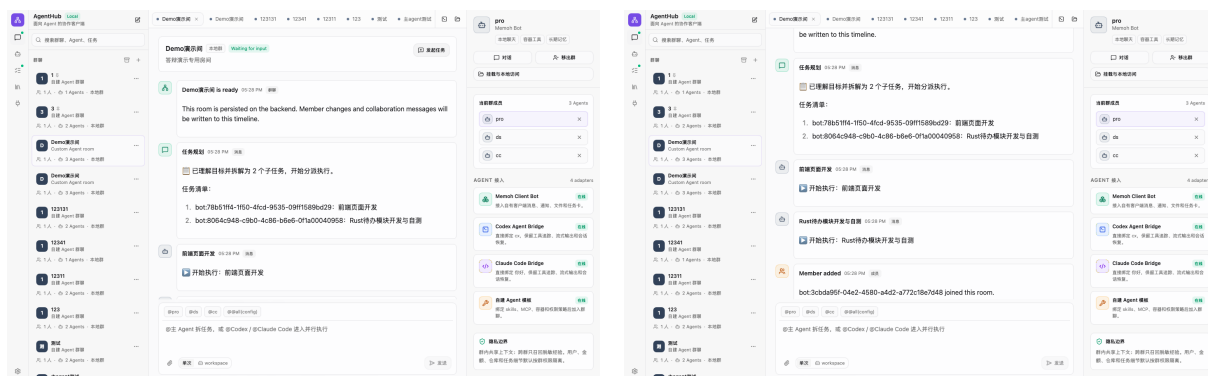


Figure 4: 左: 群聊编排「任务规划」卡片; 右: 运行中任务的实时过程气泡 (thinking / 工具调用)。

8 路线图 (高优架构演进方向)

P1(差异化强化): 启用 Codex 等更多适配器提升平台多样性 → 落地 Plan 强优先交互与用户确认闸 → 推进接口契约文档机制 (生成 / 传递 / 校验)。

P2(技术纵深): 探索多级 Sub-Orchestrator 与预评估路由 → 一键部署发布生态 (预览 URL / 状态卡) → 完善产物版本历史与对话式精细修改 → 强化通信层 (群聊 WS 全量推送) → 前沿安全与资源体系建设 (群聊 Token balance、Markov 监控、精细化 token 预算)。

A 本文档引用的研究

主题	文献
多轮对话退化	Laban et al., Microsoft, arXiv:2505.06120
单 Agent 等预算不弱于多 Agent	Tran & Kiela, Stanford, arXiv:2604.02460
多 Agent token 成本	Lin et al., arXiv:2505.18286
Safety Drift 可预测	Dhodapkar & Pishori, arXiv:2603.27148(预印本)
多 Agent 失败模式分类	Cemri et al., UC Berkeley, arXiv:2503.13657
契约 + 轨迹框架 (MAT)	Paduraru et al., arXiv:2603.18096(预印本)
上下文腐化	Liu et al., arXiv:2307.03172;Chroma Research
迭代自我精炼 / 反思	Madaan et al., arXiv:2303.17651;Shinn et al., arXiv:2303.11366

标注为预印本者尚未经同行评审; 所有数值为设计参考基准, 本产品的实现将在落地相应机制时独立验证。